

CSCI 22042 - Visual Programming

Individual Assignment

Bachelor of Science Honours in Computer Science
Faculty of Computing and Technology, University of Kelaniya
Academic Year 2024/2025 - Semester II

Assignment Overview

In this assignment, you will individually design and implement a **desktop algorithm visualizer application** using **C# and Windows Forms (.NET)**. The application is a multi-form WinForms desktop tool that allows a user to visually observe the step-by-step execution of sorting and pathfinding algorithms through animated, interactive graphical interfaces.

Before beginning development, read this handout carefully and in full. Pay close attention to the **Mandatory Features**, **Technical Requirements**, and **Algorithm Constraints** sections, as all of these form part of your assessment.

You have full freedom to choose your own UI layouts, colour schemes, and visual design approaches, as long as the required features are correctly implemented and the technical constraints outlined below are respected.

This is an **individual** assignment. All design and implementation work must be entirely your own.

Key Date

Milestone	Due Date
Final submission (PDF + repository ZIP + demo video)	End of Week 8

Assessment Breakdown

Component	Marks
Sorting Visualizer	35%
Pathfinding Visualizer	35%
Multi-Form Structure and Navigation	15%
GitHub Workflow	10%
Code Quality and Validation	5%
Total	100%

Algorithm Constraints

Sorting Algorithms

You must implement **at least 2** sorting algorithms. You must choose from the following permitted list only:

- Insertion Sort
- Merge Sort
- Quick Sort
- Shell Sort
- Heap Sort

Note: Bubble Sort and Selection Sort are **not permitted**. Submissions that include either of these as one of the two mandatory algorithms will not receive marks for that algorithm slot.

Pathfinding Algorithms

You must implement **exactly 1** pathfinding algorithm. You must choose from the following permitted list only:

- Breadth-First Search (BFS)
- Dijkstra's Algorithm
- Greedy Best-First Search
- A* (A-Star)

Note: Depth-First Search (DFS) is **not permitted** as the chosen pathfinding algorithm.

Algorithm Accuracy

The animation must faithfully represent the actual steps of the chosen algorithm. Comparisons, swaps, and node expansions must reflect the real execution, not a simplified or approximated version. This will be assessed during the demo video review.

Merge Sort note: Merge Sort operates on subarrays. Your animation must visually reflect the divide and merge phases accurately. Animating it as a simple swap-based sort will not be accepted.

Mandatory Features

All features listed in this section are mandatory. Failure to implement a mandatory feature will result in the loss of marks allocated to that component.

Multi-Form Application Structure

Your application must consist of **at least the following four forms**:

- **Main Form:** The launch screen of the application. Provides navigation to the Sorting Visualizer and the Pathfinding Visualizer.
- **Sorting Visualizer Form:** Contains the full sorting animation interface.
- **Pathfinding Visualizer Form:** Contains the full pathfinding animation interface.
- **Settings Form:** Opened modally from both visualizer forms. Allows the user to configure parameters for the active visualizer. Configured values must be correctly passed back to and applied by the calling form.

Sorting Visualizer

The Sorting Visualizer form must satisfy the following requirements:

- Visually represent array elements using a bar chart drawn on a **Panel** control, where bar height corresponds to element value.
- Implement at least 2 sorting algorithms from the permitted list (Section 4.1).
- Animate the sorting process step by step. Each step must correspond to one logical operation of the algorithm (one comparison or one swap/shift).
- Use distinct colours to indicate the state of each bar at any point during the animation (e.g. default, being compared, being swapped, fully sorted).
- Display a live comparison counter that updates in real time during the animation.
- Allow the user to generate a new random array.
- Provide controls to start and reset the animation. Controls must be appropriately enabled or disabled based on the current application state.
- Array size and animation speed must be configurable through the Settings Form.

Pathfinding Visualizer

The Pathfinding Visualizer form must satisfy the following requirements:

- Display an interactive grid of tiles drawn on a **Panel** control. Grid size must be configurable through the Settings Form.
- Allow the user to place a start tile and an end tile on the grid.
- Allow the user to draw wall tiles on the grid. Walls are impassable nodes.
- Implement exactly 1 pathfinding algorithm from the permitted list (Section 4.2).
- Animate the algorithm's traversal step by step, highlighting visited nodes progressively as the algorithm expands them.
- Upon completion, highlight the final path from start to end in a distinct colour.
- Use distinct colours for the following tile states: unvisited, wall, start, end, visited/explored, and final path.
- Provide controls to start and clear the grid. Controls must be appropriately enabled or disabled based on the current application state.
- Attempting to start the visualizer without a valid start and end tile placed must produce a clear error message.
- Animation speed must be configurable through the Settings Form.

Technical Requirements

- **Language and Platform:** C#, Windows Forms (.NET)
- **IDE:** Visual Studio
- **Drawing:** All bar chart and grid rendering must use GDI+ (`System.Drawing`) via the `Paint` event of a **Panel** control. Third-party visualization or charting libraries are **not permitted** for the visualizer panels.
- **Animation:** All step-by-step animation must be driven by a `System.Windows.Forms.Timer`. Busy-wait loops or `Thread.Sleep` on the UI thread are **not permitted**.
- **Algorithm Implementation:** All sorting and pathfinding logic must be implemented from scratch. Built-in methods such as `Array.Sort()` or `List.Sort()` must **not** be used as the algorithm implementation.
- **OOP:** Each algorithm must be encapsulated in its own class. Implementing all

algorithm logic inline within a form's code-behind is not acceptable.

- **Validation and Error Handling:** All user inputs must be validated. The application must not crash or produce unhandled exceptions under normal use. All **Timer**-driven animations must stop cleanly when a form is closed or reset.

GitHub Workflow (10%)

Proper use of Git and GitHub is a **graded component** of this assignment. Your repository history will be reviewed as part of the assessment.

Repository Setup

- Create one **private GitHub repository** for your project.
- The repository must be initialised with a proper `.gitignore` (use the Visual Studio template).
- Add the LIC as a collaborator before the submission deadline.
- **LIC GitHub username:** @haturusinghe

Branching and Pull Requests

- The **main** branch must always contain a working, stable version of the application.
- All development work must be done on **feature branches** (e.g. `feature/sorting-visualizer`, `feature/pathfinding-grid`).
- Merging into **main** must be done via **Pull Requests (PRs)**.
- ***Direct commits to main are not permitted after the initial setup.***

Commit Practices

- Make **regular, meaningful commits** throughout the development period. A commit history consisting of only a few large commits will not receive full marks.
- Commit messages must be descriptive (e.g. `Implement QuickSort step iterator with pivot highlighting`, not `update` or `fix stuff`).
- Commits should be appropriately sized. Avoid committing large volumes of unrelated changes in a single commit.

Deliverables

Submit the following via Moodle before the deadline. All three items must be submitted together in a single Moodle submission.

1. Submission PDF

- A single PDF file containing your **full name, student registration number**, and **course code (CSCI 22042)**.
- The PDF must include the following links clearly labelled:
 - Your **GitHub repository URL**
 - Your **demo video URL**
- This PDF is your submission record. Submissions without a valid PDF will not be assessed.

2. GitHub Repository ZIP

- A **ZIP export of your GitHub repository** containing the complete Visual Studio project and all source files.
- The ZIP must be exported directly from GitHub (Code → Download ZIP) so that the full project structure is preserved.
- The repository must include a **README** in the root with brief setup and run instructions.
- The LIC must have been added as a collaborator on the GitHub repository prior to submission.
- **LIC GitHub username:** @haturusinghe

3. Screen-Recorded Demo Video

- A screen recording of the running application demonstrating all implemented mandatory features.
- The video must show: algorithm selection, the Settings Form in use, both visualizers running with full animation, wall drawing and pathfinding on the grid, and the final path highlighted.
- The recording must be clear and unedited. Maximum duration: **5 minutes**.
- Upload the video to a file sharing or video hosting platform (e.g. Google Drive, YouTube unlisted) and include the link in the submission PDF.

Academic Integrity and Late Submission Policy

Late Submissions: Late submissions will be penalised **10% per day**, including weekends. Submissions more than three days late will not be accepted.

Academic Integrity: All work submitted must be entirely your own. You may consult online resources, documentation, and tutorials for learning purposes, but you must not copy code directly from any source, including other students, GitHub repositories, or AI-generated code. Any external references used must be acknowledged in your README. Submitting code written by others without attribution constitutes academic dishonesty and will be handled in accordance with the University of Kelaniya academic integrity policy (CQA/A/P/03).

Questions and Support

Direct all assignment-related questions to the LIC:

- **Email:** shatur251@kln.ac.lk